

PhytoNexus Web Architecture

FRONTEND IMPLEMENTATION TEMPLATE // DEMO MODE

1. Website Overview & Demo Constraints

Demo Mode Constraint: For the ideathon presentation, this website will operate in an isolated prototype state. External hardware connections (ESP32/Wi-Fi) are decoupled. All live dashboard data, charts, and events must be driven by a local JavaScript simulation script (mocking telemetry over time). The UI must behave exactly as if it were receiving real-time JSON payloads from the physical botanical extraction setup.

The platform serves as a unified digital hub for small-scale botanical extraction R&D. Its primary frontend goal is to translate raw hardware sensor readings into structured, traceable research records while clearly surfacing safety states.

2. Implemented Telemetry Parameters

To maintain strict alignment with the existing prototype BOM, the frontend data schema must only expect and process the following specific variables. Do not build UI components for sensors that do not exist in the current reference stack.

Variable Name	Hardware	Data Type	UI Role in Botanical R&D
temperature_c	DHT22	Float (°C)	Primary process variable. Plotted on main timeline. High values trigger Warning/Alert states for solvent safety.
humidity_rh	DHT22	Float (%)	Ambient/chamber conditions. Used as secondary metadata in experiment records to ensure environmental reproducibility.
vapor_signal	MQ-3	Integer (Raw)	Alcohol-vapor proxy signal. Treated as a hazard indicator for extraction leaks, not a calibrated PPM concentration.
heater_state	5V Relay	Boolean (0/1)	Electrical load status. Displayed as a binary ON/OFF indicator. Automatically forced to 0 during Alert/Fault conditions.

3. System States & UI Color Coding

The website must visually communicate the safety state of the extraction process at all times. Use the following color-coded UI schema across badges, borders, and notification banners based on the mocked `system_state` variable.

NORMAL (Green / #10b981)

All variables (temp, vapor) within safe operating envelope. System monitoring and logging actively. Heater allowed to operate.

WARNING (Yellow / #eab308)

Approaching threshold limit (e.g., rising vapor proxy). UI triggers soft notification. Operation continues but is flagged in timeline.

ALERT (Orange / #f97316)

Critical limit exceeded. The UI must show `heater_state` forced to OFF. Requires manual simulated acknowledgement to clear.

FAULT (Red / #ef4444)

Simulated hardware failure (e.g., DHT22 disconnected). Missing/implausible data. UI obscures live values with a "Data Invalid" overlay.

LOCK (Purple / #7c3aed)

Repeated critical faults. System disabled. UI displays a full-screen or modal block requiring a deliberate "Reset Protocol" interaction.

4. Point-Wise Site Architecture

The frontend application should be divided into the following discrete views, optimized for laboratory research use:

4.1 Home / Live Dashboard

- > **Status Banner:** Prominent display of the current System State (Normal, Warning, etc.) utilizing the color coding above.
- > **Live Metrics Cards:** Three large numeric readouts for `temperature_c`, `humidity_rh`, and `vapor_signal`. Include a smaller indicator showing the `heater_state` (ON/OFF).
- > **Real-Time Timeline Chart:** A mocked scrolling line chart (using Chart.js, Recharts, or pure SVG) plotting Temperature and Vapor Signal against Time.
- > **Event Log Feed:** A scrolling sidebar showing simulated timestamped events (e.g., "14:02:05 - Relay engaged", "14:15:22 - WARNING: Temp approaching limit").

4.3 Hardware Configuration

- > **Profile Selector:** Dropdown to select "Reference Stack V1 (ESP32 + DHT22 + MQ-3)". (Other profiles disabled for demo).
- > **Variable Mapping Table:** A visual matrix showing how physical pins map to logical software variables, proving the "Hardware-Agnostic" claim to judges.
- > **Threshold Settings:** Input sliders for users to define the simulated Warning and Alert limits for Temperature and Vapor Signal.

4.4 Analytics & History

- > **Run Comparison:** A static, hardcoded view showing an overlay of two previous extraction runs (e.g., "Run A - Successful" vs "Run B - Alert Triggered").
- > **Process Summary:** Calculated metrics derived from the run (Total Run Time, Max Temp Reached, Heater Active Time).
- > **Export Function:** A button that downloads a static mock CSV file demonstrating how structured data is generated for further laboratory analysis.

4.5 Documentation & Onboarding

- > **Wokwi Simulator Link:** Embedded iframe or direct link to the reference Wokwi project, proving the system can be verified virtually before physical assembly.
- > **Assembly Reference:** Static diagrams or text summarizing the BOM and physical wiring instructions for grassroots innovators.
- > **Safety Disclaimers:** Prominent text reinforcing that the platform manages R&D data and local logic, but physical chemical safety/ventilation remains a laboratory requirement.

5. Frontend Demo Implementation Notes

To successfully demonstrate the platform without live hardware:

- > Create a central JavaScript interval (e.g., running every 2000ms) that generates a JSON object containing `{ temp: 35.2, hum: 40.1, vapor: 150, heater: 1, state: 'NORMAL' }`.
- > Write a "Scenario Array" that sequentially updates this mock data to intentionally force the UI through the Warning, Alert, and Fault states over a 2-3 minute demo.
- > Ensure the UI reacts immediately to these state changes (changing banner colors, appending to the log, shutting off the mocked relay) to prove the safety logic works at the software level.